

Requirements Analysis and Specification

Prof. R. Mall

Dept. of CSE, IIT, Kharagpur

Requirements Analysis and Specification

- Many projects have failed:
 - Because they started to implement the software.
 - Without ever determining whether they are building what the customer really wanted.

What are Requirements?

- A Requirement is:

- A capability or condition required from the system.

- What is involved in requirements analysis and specification?

- Determine what is expected by the client from the system.
 - Document them in the form that is clear to the client as well as to the development team members.

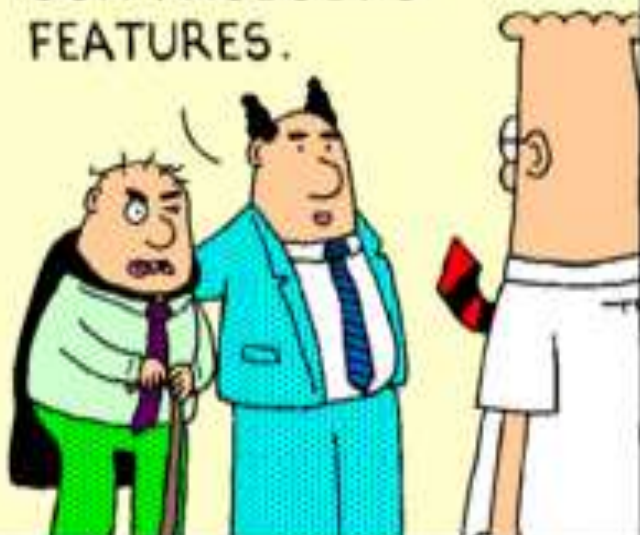
Understand and specifying requirements

- **A problem of scale:**
 - For small scale: understanding and specifying requirements is easy
 - **For large scale:** very hard; probably the hardest, most problematic and error prone development task
- **The requirements specification task:**
 - Input: User needs in minds of people (hopefully)!
 - Output: precise statement of what the future system will do

Requirements for Products

- When a company plans to develop a generic product:
 - Who gives the requirements?
- The sales personnel

I HIRED A CREEP
TO HELP DETERMINE
OUR PRODUCT'S
FEATURES.



YOU
NEED
MORE
FEATURES.

GOOD
WORK.



WHEN
CAN YOU
HAVE
THAT
DONE?

GAAA!!



THE FEATURE CREEP

IS IT TOO LATE
TO GIVE OUR
PRODUCT A
LOW-BATTERY
INDICATOR?



I'D HAVE TO WORK
NIGHT AND DAY FOR
A MONTH! MY
HEALTH WOULD
DECLINE AND I'D
MISS ALL MY
OBJECTIVES!



I JUST REALIZED
THAT OTHER
PEOPLE'S PROBLEMS
MAKE ME ALL
WARM INSIDE.



Dilbert: Scott Adams

WE CAN'T
COMPETE
ON PRICE.



www.dilbert.com scottadams@aol.com

www.dilbert.com

WE ALSO CAN'T
COMPETE ON QUALITY,
FEATURES OR SERVICE.



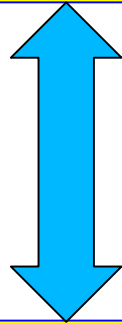
9-13-07 ©2007 Scott Adams, Inc./Distr. by UFS, Inc.

THAT LEAVES FRAUD,
WHICH I'D LIKE YOU
TO CALL MARKETING.

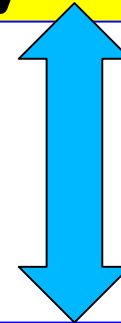


Activities in Requirements Analysis and Specification

Requirements Gathering

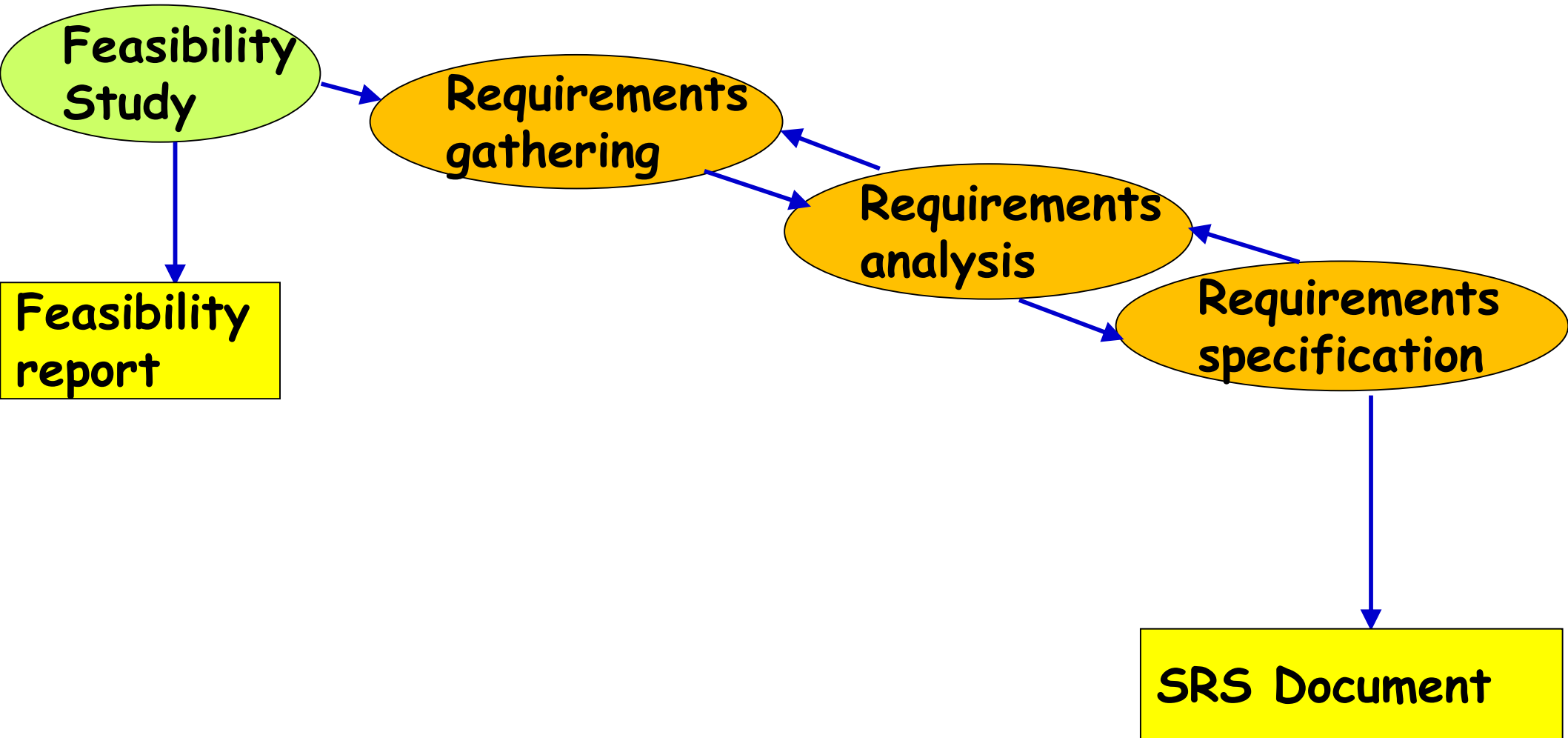


Requirements Analysis



Requirements Specification

Requirements Engineering Process



WALLY, WE DON'T HAVE
TIME TO GATHER THE
PRODUCT REQUIRE-
MENTS AHEAD OF
TIME.



www.unitedmedia.com

S. Adams

I WANT YOU TO START
DESIGNING THE
PRODUCT ANYWAY.
OTHERWISE IT WILL
LOOK LIKE WE AREN'T
ACCOMPLISHING ANY-
THING.



5/9/97 © 1997 United Feature Syndicate, Inc.

OF ALL MY PROJECTS,
I LIKE THE DOOMED
ONES BEST.



Requirements Analysis and Specification

- Requirements Gathering:
 - Fully understand the user requirements.
- Requirements Analysis:
 - Remove inconsistencies, anomalies, etc. from requirements.
- Requirements Specification:
 - Document requirements properly in an SRS document.

I'LL DESIGN THE
SYSTEM AS SOON AS
YOU GIVE ME THE
USER REQUIREMENTS.



BETTER YET, YOU
COULD BUILD THE
SYSTEM, THEN I'LL
TELL YOUR BOSS THAT
IT DOESN'T MEET MY
NEEDS.



I DON'T MEAN TO
FRIGHTEN YOU, BUT
YOU'LL HAVE TO DO
SOME ACTUAL WORK.



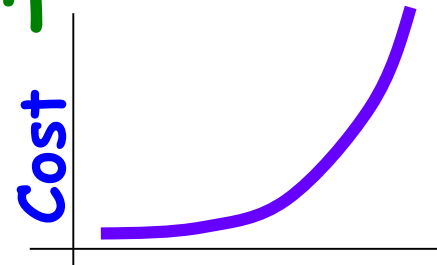
© 2003 United Feature Syndicate, Inc.

Need for SRS...

- **Good SRS reduces development cost**
 - Req. errors are expensive to fix later
 - Req. changes cost a lot (typically 40%)
 - Good SRS can minimize changes and errors
 - **Substantial savings --- effort spent during req. saves multiple times that effort**

- **An Example:**

- Cost of fixing errors in req. , design , coding , acceptance testing and operation are 2 , 5 , 15 , 50 , 150 person-months



Uses of SRS Document

- Establish the basis for agreement between the customers and the suppliers
- Provides starting point for development.
- Provide a basis for estimating costs and schedules.
- Provide a baseline for validation and verification.
- Facilitates transfer.
- Serves as a basis for enhancement.

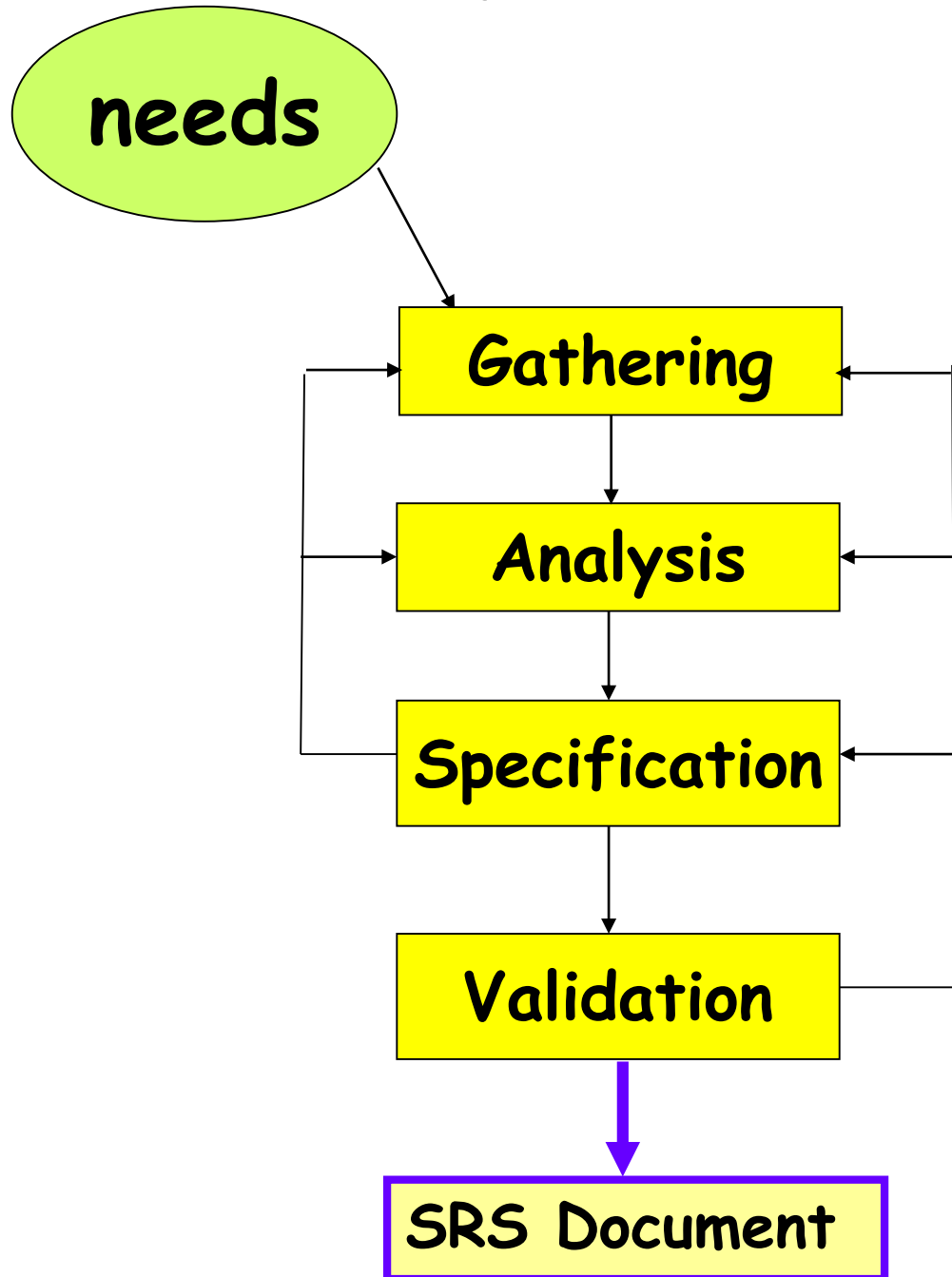
Forms Basis for User Manual

- The SRS can serve as the basis for writing User Manual for the software:
 - User Manual: Describes the functionality from the perspective of a user --- An important document for users.
 - Typically also describes how to carry out the required tasks with examples.

SRS Document: Stakeholders

- SRS intended for a diverse audience:
 - Customers and users for validation, contract, ...
 - Systems (requirements) analysts
 - Developers, programmers to implement the system
 - Testers to check that the requirements have been met
 - Project Managers to measure and control the project
- Different levels of detail and formality is needed for each audience
- Different templates for requirements specifications:
 - Often variations of **IEEE 830**

Requirement process..



- Specification itself may lead to more analysis
- Validation can show gaps that can lead to further analysis and specification .

SHOULD I CONTINUE
TO MANAGE ISSUES?

OR SHOULD I ALIGN
ORGANIZATIONAL
ACTIVITIES WITH
STAKEHOLDER
EXPECTATIONS?

WHICH
ANSWER
WOULD
CAUSE YOU
TO DO REAL
WORK?

WHAT IS
THIS, A
FARM?

DilbertCartoonist@gmail.com

© 2011 Scott Adams, Inc./Dist. by UFS, Inc.

How to Gather Requirements?

- Observe existing (manual) systems,
- Study existing procedures,
- Discuss with customer and end-users,
- Analyse what needs to be done
- Input and Output analysis

Requirements Gathering Activities

- 1. Study existing documentation
- 2. Interview
- 3. Task analysis
- 4. Scenario analysis
- 5. Form analysis

WE INTERVIEWED
HUNDREDS OF USERS
AND TURNED ALL OF
THEIR SUGGESTIONS
INTO FEATURES.



AS IT TURNS OUT,
EVERY USER WE
TALKED TO WAS AN
IDIOT, AND THEIR
DUMB SUGGESTIONS
RUINED OUR PRODUCT.



IN HINDSIGHT, WE
PROBABLY SHOULD HAVE
TALKED TO PEOPLE
WHO WORK OUTSIDE
THIS BUILDING.



Dilbert.com DilbertCartoonist@gmail.com

5-7-12 ©2012 Scott Adams, Inc. Dist. by Universal Uclick

Requirements Gathering (CONT.)

- In the absence of a working system,
 - Lot of imagination and creativity are required.
- Interacting with the customer to gather relevant data:
 - Requires a lot of experience.

Requirements Gathering (CONT.)

- Some desirable attributes of a good system analyst:
 - Good interaction skills,
 - Imagination and creativity,
 - Experience...

TODAY IS THE THREE-YEAR ANNIVERSARY OF OUR FIRST MEETING TO DISCUSS PROJECT REQUIREMENTS.



scottadams@aol.com

www.dilbert.com

AND WE'RE STILL DISCUSSING REQUIREMENTS. DOES ANYONE ELSE SEE A PROBLEM HERE?



4/14/02 © 2002 United Feature Syndicate, Inc.

WHEN YOU'RE DONE, CAN WE TALK ABOUT REQUIREMENTS?



Case Study: Automation of Office Work at CSE Dept.

- The academic, inventory, and financial information at the CSE department:
 - Being carried through manual processing by two office clerks, a store keeper, and two attendants.
- Considering the low budget he had at his disposal:
 - The HoD entrusted the work to a team of student volunteers.

Case Study: Automation of Office Work at CSE Dept.

- The team was first briefed by the HoD:
 - Concerning the specific activities to be automated.
- The analysts first discussed with the two office clerks: **Interview**
 - Regarding their specific responsibilities (tasks) that were to be automated.
- The analyst also interviewed student and faculty representatives who would also use the software.

Case Study: Automation of Office Work at CSE Dept.

- For each task that a user needs the software to perform, they asked:
 - The steps through which these are to be performed. **Task and Scenario Analysis**
 - The various scenarios that might arise for each task.
- The analyst also collected the different types of forms that were being used. **Form Analysis**

Case Study: Automation of Office Work at CSE Dept.

- The analysts understood the requirements for the system from various user groups:
 - Identified inconsistencies, ambiguities, incompleteness.
- Resolved the requirements problems through discussions with users:
 - Resolved a few issues which the users were unable to resolve through discussion with the HoD.
- Documented the requirements in the form of an SRS document.

Requirements Analysis

Requirements Specification

Analysis of Gathered Requirements

- Main purpose of req. analysis:
 - Clearly understand user requirements,
 - Detect inconsistencies, ambiguities, and incompleteness.
- Incompleteness and inconsistencies:
 - Resolved through further discussions with the end-users and the customers.

Ambiguity

“All customers must have the same control field.”

Do you notice any problems?
Multiple interpretations possible

1. All customers have the same value in their control field.
2. All control fields have the same format.
3. One control field is issued for all customers.

Inconsistent Requirement

- Some part of the requirement:
 - contradicts with some other part.
- Example:
 - One customer says turn off heater and open water shower when temperature $> 100^{\circ}\text{C}$
 - Another customer says turn off heater and turn ON cooler when temperature $> 100^{\circ}\text{C}$

Incomplete Requirement

- Some requirements not included:
 - Possibly due to oversight.
- Example:
 - The analyst has not recorded:
when temperature falls below 90°C
 - heater should be turned ON
 - water shower turned OFF.

Analysis of the Gathered Requirements (CONT.)

- Requirements analysis involves:
 - Obtaining a clear, in-depth understanding of the software to be developed,
 - Remove all ambiguities and inconsistencies from the initial customer perception of the problem.

Analysis of the Gathered Requirements (CONT.)

- It is quite difficult to obtain:
 - A clear, in-depth understanding of the problem:
 - Especially if there is no working model of the problem.

Analysis of the Gathered Requirements (CONT.)

- Experienced analysts take considerable time:
 - Clearly understand the exact requirements the customer has in his mind.

Analysis of the Gathered Requirements (CONT.)

- Experienced systems analysts know - often as a result of painful experiences ---
 - “Without a clear understanding of the problem, it is impossible to develop a satisfactory system.”

Analysis of the Gathered Requirements (CONT.)

- Several things about the project should be clearly understood by the analyst:
 - What is the problem?
 - Why is it important to solve the problem?
 - What are the possible solutions to the problem?
 - What complexities might arise while solving the problem?

Analysis of the Gathered Requirements (CONT.)

- Some anomalies and inconsistencies can be very subtle:
 - Escape even most experienced eyes.
 - If a formal model of the system is constructed,
 - Many of the subtle anomalies and inconsistencies get detected.

Analysis of the Gathered Requirements_(CONT.)

- After collecting all data regarding the system to be developed,
 - Remove all inconsistencies and anomalies from the requirements,
 - Systematically organize requirements into a Software Requirements Specification (SRS) document.

Software Requirements Specification

- Main aim of requirements specification:
 - Systematically organize the requirements arrived during requirements analysis.
 - Document requirements properly.

SRS Document

- As already pointed out---
useful in various contexts:
 - Statement of user needs
 - Contract document
 - Reference document
 - Definition for implementation

SRS Document (CONT.)

- The SRS document is known as **black-box specification**:
 - The system is considered as a black box whose internal details are not known.
 - Only its visible external (i.e. input/output) behavior is documented.



SRS Document (CONT.)

- SRS document concentrates on:
 - **What** needs to be done in terms of input-output behaviour
 - Carefully avoids the solution ("**how to do**") aspects.

SRS Document (CONT.)

- The requirements at this stage:
 - Written using end-user terminology.
- If necessary:
 - Later a formal requirement specification may be developed from it.

Properties of a Good SRS Document

- It should be concise
 - and at the same time should not be ambiguous.
- It should specify what the system must do
 - and not say how to do it.
- Easy to change.,
 - i.e. it should be well-structured.
- It should be consistent.
- It should be complete.

Properties of a Good SRS Document

(cont...)

- It should be traceable
 - You should be able to trace which part of the specification corresponds to which part of the design, code, etc and vice versa.
- It should be verifiable
 - e.g. "system should be user friendly" is not verifiable

SRS should not include...

- **Project development plans**

- E.g. cost, staffing, schedules, methods, tools, etc
 - Lifetime of SRS is until the software is made obsolete
 - Lifetime of development plans is much shorter

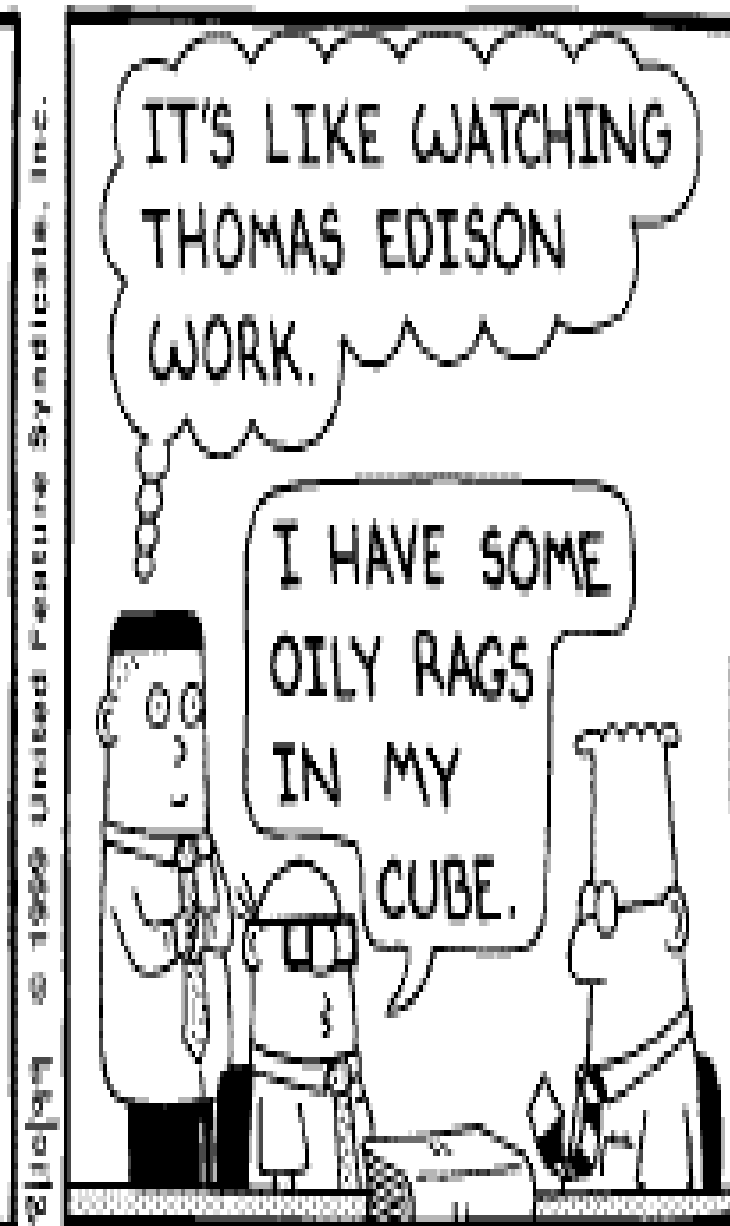
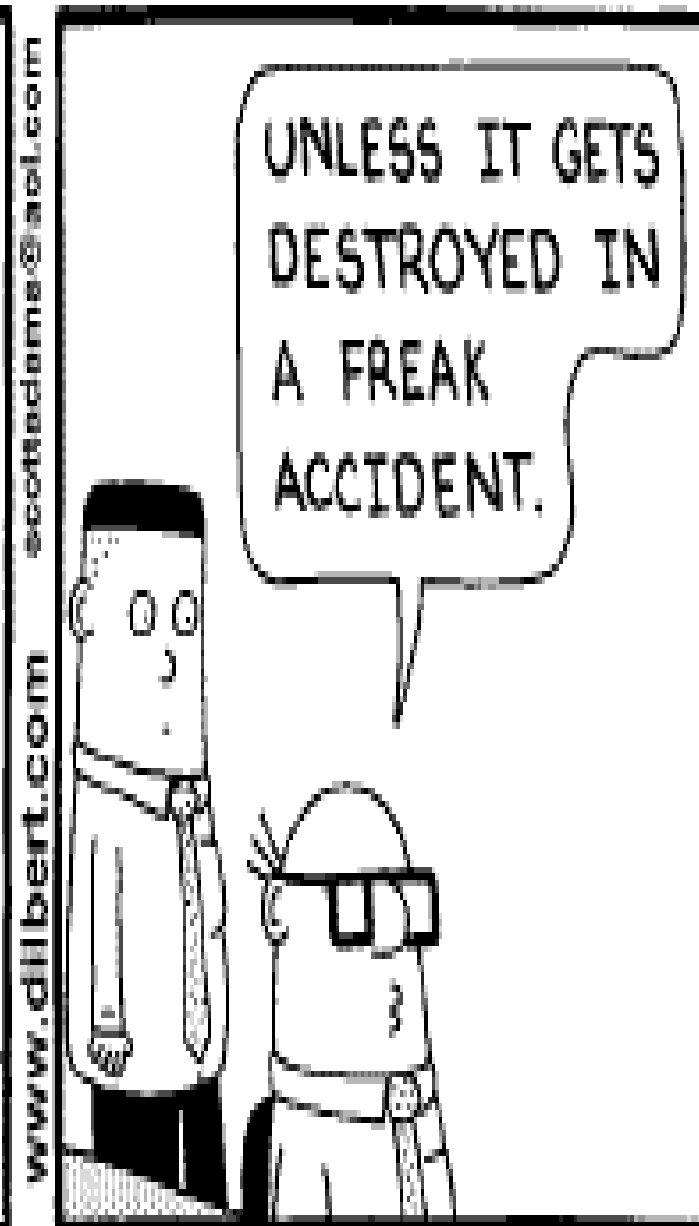
- **Product assurance plans**

- Configuration Management, Verification & Validation, test plans, Quality Assurance, etc
 - Different audiences
 - Different lifetimes

- **Designs**

- Requirements and designs have different audiences
- Analysis and design are different areas of expertise

A Badly Written Gigantic Document is a Pain to Users



Dilbert: Scott Adams

SRS Document (CONT.)

- Following are possibly the most important parts:
 - Functional requirements,
 - External Interfaces
 - Non-functional requirements,
 - Constraints

Functional Requirements

- Specifies all the functionality that the system should support
 - Heart of the SRS document:
 - Forms the bulk of the Document
- Outputs for the given inputs and the relationship between them
- Must specify behavior for invalid inputs too!

Functional Requirement Documentation

- **Overview**

- describe purpose of the function and the approaches and techniques employed

- **Inputs and Outputs**

- sources of inputs and destination of outputs
- quantities, units of measure, ranges of valid inputs and outputs
- timing

Functional Requirement Documentation

• Processing

- validation of input data
- exact sequence of operations
- responses to abnormal situations
- any methods (eg. equations, algorithms) to be used to transform inputs to outputs

Nonfunctional Requirements

- Characteristics of the system which can not be expressed as functions:
 - Maintainability,
 - Portability,
 - Usability,
 - Security,
 - Safety, etc.

Nonfunctional Requirements

- Reliability issues
- Performance issues:
 - Example: How fast the system can produce results
 - so that it does not overload another system to which it supplies data, etc.
 - Needs to be measurable (verifiability)
 - Eg resp time should be xx 90% of the time

Constraints

- Hardware to be used,
- Operating system
 - or DBMS to be used
- Capabilities of I/O devices
- Standards compliance
- Data representations
 - by the interfaced system

External Interface Requirements

- User interfaces
- Hardware interfaces
- Software interfaces
- Communications interfaces with other systems
- File export formats

Goals of Implementation

- Goals describe things that are desirable of the system:
 - But, would not be checked for compliance.
 - For example,
 - Reusability issues
 - Functionalities to be developed in future

IEEE 830-1998 Standard for SRS

- Title
 - Table of Contents
 - 1. Introduction
 - 1.1 Purpose
 - 1.2 Scope
 - 1.3 Definitions, Acronyms, and Abbreviations
 - 1.4 References
 - 1.5 Overview
 - 2. Overall Description
 - 3. **Specific Requirements**
 - Appendices
 - Index
-
- The diagram consists of five yellow rectangular callout boxes with black borders, each containing a list of requirements. Arrows point from these boxes to specific sections in the main list:
- Callout 1 (top right):** Points to "1.1 Purpose".
 - Describe purpose of this SRS
 - Describe intended audience
 - Callout 2 (middle right):** Points to "1.2 Scope".
 - Identify the software product
 - Enumerate what the system will and will not do
 - Describe user classes and benefits for each
 - Callout 3 (middle right):** Points to "1.4 References".
 - Define the vocabulary of the SRS (may reference appendix)
 - Callout 4 (bottom right):** Points to "1.5 Overview".
 - List all referenced documents including sources (e.g., Use Case Model and Problem Statement; Experts in the field)
 - Callout 5 (bottom right):** Points to "3. Specific Requirements".
 - Describe the content of the rest of the SRS
 - Describe how the SRS is organized

IEEE 830-1998 Standard - Section 2 of SRS

- Title

- Table of Contents

- Present the business case and operational concept of the system
- Describe how the proposed system fits into the business context
- Describe external interfaces: system, user, hardware, software, communication
- Describe constraints: memory, operational, site adaptation

- 1. Introduction

- 2. Overall Description

- 2.1 Product Perspective

- Summarize the major functional capabilities
- Include the Use Case Diagram and supporting narrative (identify actors and use cases)
- Include Data Flow Diagram if appropriate

- 2.2 Product Functions

- 2.3 User Characteristics

- Describe and justify technical skills and capabilities of each user class

- 2.4 Constraints

- 2.5 Assumptions and Dependencies

- 3. Specific Requirements

- Describe other constraints that will limit developer's options; e.g., regulatory policies; target platform, database, network software and protocols, development standards requirements

- 4. Appendices

- 5. Index

IEEE 830-1998 Standard - Section 3 of SRS (1)

- ...
- 1. Introduction
- 2. Overall Description
- 3. Specific Requirements
 - 3.1 External Interfaces
 - 3.2 Functions
 - 3.3 Performance Requirements
 - 3.4 Logical Database Requirements
 - 3.5 Design Constraints
 - 3.6 Software System Quality Requirements
 - 3.7 Object Oriented Models
- 4. Appendices
- 5. Index

Specify software requirements in sufficient detail to enable designers to design a system to satisfy those requirements and testers to verify requirements

State requirements that are externally perceivable by users, operators, or externally connected systems

Requirements should include, at a minimum, a description of every input (stimulus) into the system, every output (response) from the system, and all functions performed by the system in response to an input or in support of an output

- (a) Requirements should have characteristics of high quality requirements
- (b) Requirements should be cross-referenced to their source.
- (c) Requirements should be uniquely identifiable
- (d) Requirements should be organized to maximize readability

IEEE 830-1998 Standard - Section 3 of SRS (2)

- ...

- 1. Introduction

- 2. Overall Description

- 3. Specific Requirements

- 3.1 External Interfaces

- 3.2 Functions

- 3.3 Performance Requirements

- 3.4 Logical Database Requirements

- 3.5 Design Constraints

- 3.6 Software System Quality Attributes

- 3.7 Object Oriented Models

- 4. Appendices

- 5. Index

•Detail all inputs and outputs
(complement, not duplicate, information presented in section 2)
•Examples: GUI screens, file formats

•Include detailed specifications of each
use case, including collaboration and
other diagrams useful for this purpose

•Include:
a) Types of information used
b) Data entities and their relationships

•Should include:
a) Standards compliance
b) Accounting & Auditing procedures

•The main body of requirements organized in a variety of
possible ways:
a) Architecture Specification
b) Class Diagram
c) State and Collaboration Diagrams
d) Activity Diagram (concurrent/distributed)

IEEE 830-1998 Standard - Templates

- Section 3 (Specific Requirements) may be organized in many different ways based on
 - Modes
 - User classes
 - Concepts (object/class)
 - Features
 - Stimuli
 - Organizations

SRS explained

. 1.0 INTRODUCTION

This document specifies all the requirements for

. 1.1 Purpose

The purpose of the ...is to

The system should assist

The intended audience for this document is ...

This specification describes

. 1.2 Scope

This document applies only to

This specification is not concerned with

SRS explained

2.0 GENERAL DESCRIPTION

2.1 Product Perspective

This system allows stakeholders to....

The system will display....

The system will help

The system provides information about

2.2 Product Functions

The system provides the following functions:

SRS explained

• 2.3 User Characteristics

The users of the system are:

- Level of Users' Computer Knowledge
- Level of Users' Business Knowledge
- Frequency of Use

• 2.4 General Constraints

The system will support

The system will not allow

• 2.5 Assumption and Dependencies

This system relies on

The system must have a satisfactory interface and

Section 3 of SRS

- **SPECIFIC REQUIREMENTS**

- **3.1 Functional Requirements**

- **3.1.1 Subject Registration**

- The subject registration requirements are concerned with functions regarding subject registration which includes students selecting, adding, dropping, and changing a subject.

- **F-001 (3.1.1.1):**

- The system shall allow the user to register a subject.

- **F-002 (3.1.1.2):**

- ARS shall allow the user to delete a subject if the user has chosen to drop that unit.

- **F-003 (3.1.1.3):**

- ARS shall check if a unit has been filled by enough registered students.

SRS functional Reqs

- **F-004 (3.1.1.4):**

- ARS shall allow the user to add his/her name to the unit waiting list if the user wants to register in a subject which has been filled already with enough registered students.

- **F-005 (3.1.1.5):**

- ARS shall automatically register the subject for the user who is the first one on the waiting list if a vacancy appears for that subject.

- **F-006 (3.1.1.6):**

- ARS shall allow the user to change practical session(s), if available.

Design Constraints (3.2)

- 3.2 Design Constraints
- C-001 (3.2.1):
 - ARS shall provide user interface through standard web browsers.
- C-002 (3.2.2):
 - ARS shall support PC and/or UNIX platforms.
- C-003 (3.2.3):
 - ARS shall be developed using the JAVA programming language

Non-functional requirements

- 3.3 Non-Functional Requirements

- N-001 (3.3.1):

- ARS shall respond to any retrieval in less than 5 seconds.

- N-002 (3.3.2):

- ARS shall generate a report within 1 minute.

- N-003 (3.3.3):

- ARS shall allow the user to remotely connect to the system.

- N-004 (3.3.8):

- The system will be accompanied by a well-written user manual.

Structure of an SRS

- **Introduction**

- Purpose , the basic objective of the system
- Scope of what the system is to do , not to do

- **Overall description**

- Product perspective
- Product functions
- User characteristics
- Assumptions
- Constraints

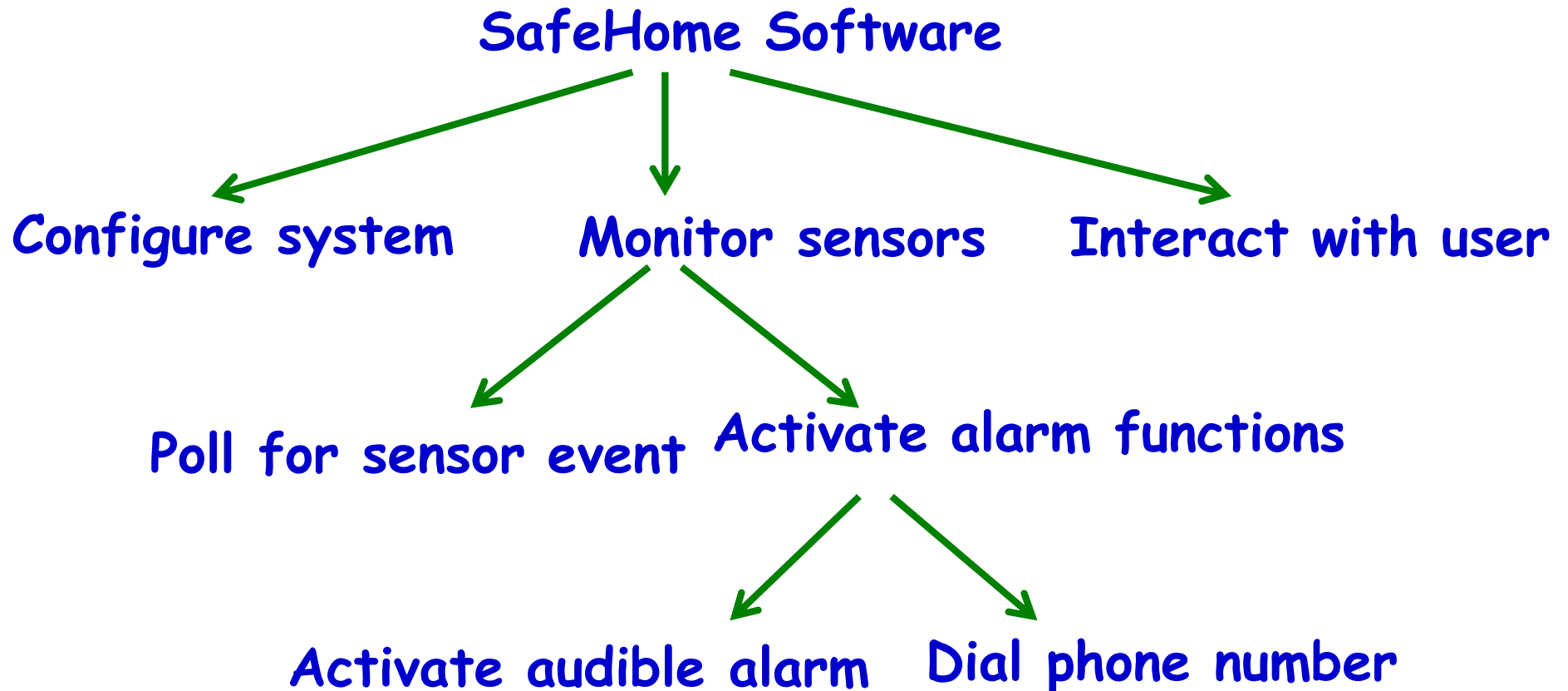
Structure of an SRS...

- **Specific requirements**
 - External interfaces
 - Functional requirements
 - Performance requirements
 - Design constraints
- **Acceptance criteria**
 - desirable to specify this up front.
- Based on IEEE 830.

Partitioning

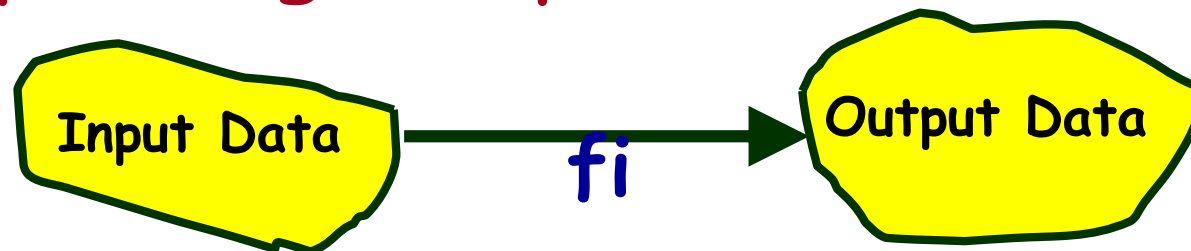
Partitioning decomposes a problem and Establishes a hierarchical representation of information (or function):

- exposing increasing detail by moving vertically in the hierarchy
- decomposing the problem by moving horizontally in the hierarchy.



Functional Requirements

- It is desirable to consider every system as:
 - Performing a set of functions $\{f_i\}$.
- Each function f_i considered as:
 - Transforming a set of input data to corresponding output data.



Example: Functional Requirement

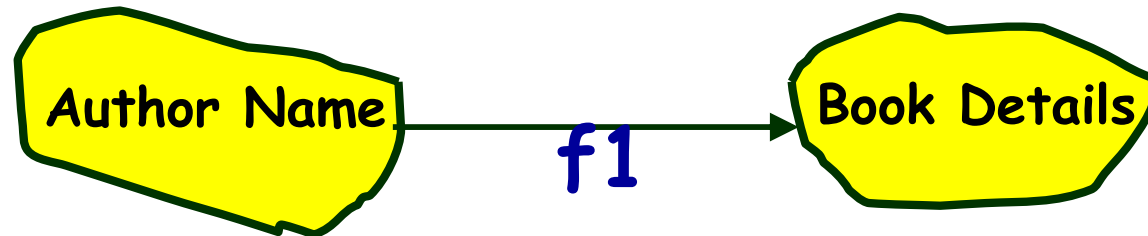
- F1: Search Book

- Input:

- an author's name:

- Output:

- details of the author's books and the locations of these books in the library.



Functional Requirements

- Functional requirements describe:
 - A set of high-level requirements
 - Each high-level requirement:
 - takes in some data from the user
 - outputs some data to the user
 - Each high-level requirement:
 - might consist of a set of identifiable functions

Functional Requirements

- For each high-level requirement:
 - A function is described in terms of:
 - Input data set
 - Output data set
 - Processing required to obtain the output data set from the input data set.

YOUR USER REQUIREMENTS INCLUDE FOUR HUNDRED FEATURES.



scottadams@aol.com

www.dilbert.com

DO YOU REALIZE THAT NO HUMAN WOULD BE ABLE TO USE A PRODUCT WITH THAT LEVEL OF COMPLEXITY?



4/14/01 © 2001 United Feature Syndicate, Inc.

GOOD POINT. I'D BETTER ADD "EASY TO USE" TO THE LIST.



Functional Requirements

- A high-level function is one:
 - Using which the user can get some useful piece of work done.
- Can the receipt printing work during withdrawal of money from an ATM:
 - Be called a high level function?
- A high-level requirement typically involves:
 - Accepting some data from the user,
 - Transforming it to the required response, and then
 - Outputting the system response to the user.

Use Cases

- A use case is a term in UML:
 - Represents a high level functional requirement.
- Use case representation is more well-defined and has agreed documentation:
 - Compared to a high-level functional requirement and its documentation
 - Therefore many organizations document the functional requirements in terms of use cases⁷⁹

Example Functional Requirements

- Req. 1:
 - Once user selects the “search” option,
 - he is asked to enter the key words.
 - The system should output details of all books
 - whose title or author name matches any of the key words entered.
 - Details include: Title, Author Name, Publisher name, Year of Publication, ISBN Number, Catalog Number, Location in the Library.

Example Functional Requirements

- Req. 2:
 - When the “renew” option is selected,
 - The user is asked to enter his membership number and password.
 - After password validation,
 - The list of the books borrowed by him are displayed.
 - The user can renew any of the books:
 - By clicking in the corresponding renew box.

High-Level Function: A Closer View

- A high-level function:
 - Usually involves a series of interactions between the system and one or more users.
- Even for the same high-level function,
 - There can be different interaction sequences (or scenarios)
 - Due to users selecting different options or entering different data items.

Req. 1:

- R.1.1:

- Input: "search" option,
- Output: user prompted to enter the key words.

- R1.2:

- Input: key words
- Output: Details of all books whose title or author name matches any of the key words.
 - Details include: Title, Author Name, Publisher name, Year of Publication, ISBN Number, Catalog Number, Location in the Library.
- Processing: Search the book list for the keywords⁸³

Req. 2:

- R2.1:

- **Input:** "renew" option selected,
- **Output:** user prompted to enter his membership number and password.

- R2.2:

- **Input:** membership number and password
- **Output:**
 - list of the books borrowed by user are displayed. User prompted to enter books to be renewed or
 - user informed about bad password
- **Processing:** Password validation, search books issued to the user from borrower list and display.

Req. 2:

- R2.3:

- **Input:** user choice for renewal of the books issued to him through mouse clicks in the corresponding renew box.
- **Output:** Confirmation of the books renewed
- **Processing:** Renew the books selected by the in the borrower list.

Examples of Bad SRS Documents

- Unstructured Specifications:
 - Narrative essay --- one of the worst types of specification document:
 - Difficult to change,
 - Difficult to be precise,
 - Difficult to be unambiguous,
 - Scope for contradictions, etc.

Examples of Bad SRS Documents

- Noise:

- Presence of text containing information irrelevant to the problem.

- Silence:

- Aspects important to proper solution of the problem are omitted.

Examples of Bad SRS Documents

- Overspecification:

- Addressing “how to” aspects
- For example, “Library member names should be stored in a sorted descending order”
- Overspecification restricts the solution space for the designer.

- Contradictions:

- Contradictions might arise
 - if the same thing described at several places in different ways.

Examples of Bad SRS Documents

- Ambiguity:

- Literary expressions
- Unquantifiable aspects, e.g. "good user interface"

- Forward References:

- References to aspects of problem
 - defined only later on in the text.

- Wishful Thinking:

- Descriptions of aspects
 - for which realistic solutions will be hard to find.

Suggestions for Writing Quality Requirements

- Keep sentences and paragraphs short.
- Use active voice.
- Use proper grammar, spelling, and punctuation.
- Use terms consistently and define them in a glossary.
- To see if a requirement statement is sufficiently well defined,
 - Read it from the developer's perspective

Suggestions for Writing Quality Requirements cont...

- Avoid stating requirements redundantly in the SRS.
 - Including the same requirement in multiple places may make the document easier to read,
 - But, it makes the document more difficult to maintain.
 - The multiple instances of the requirement all have to be updated at the same time,
 - Lest an inconsistency would creep in.

Suggestions for Writing Quality Requirements

- Split a requirement into multiple sub-requirements:
 - Because each will require separate test cases and because each should be separately traceable.
 - If several requirements are strung together in a paragraph, it is easy to overlook one during construction or testing.

Reviewing Requirements for Quality: An Example

- The product shall provide status messages at regular intervals not less than every 60 seconds.
 - What is wrong with this requirement?
- This requirement is incomplete:
 - What are the status messages and how are they supposed to be displayed to the user?
- The requirement contains ambiguities.
 - Is the interval between status messages really supposed to be at least 60 seconds, so showing a new message every 10 years is okay?

A Better Requirement

- 1. Status Messages.
- 1.1. The Background Task Manager shall display status messages in a designated area of the user interface at intervals of 60 plus or minus 10 seconds.
- 1.2. If background task processing is progressing normally, the percentage of the background task processing that has been completed shall be displayed.
- 1.3. A message shall be displayed when the background task is completed.
- 1.4. An error message shall be displayed if the background task has stalled.

SRS Review

- Review done by the Developers along with the user representatives.
- To verify that SRS confirms to the actual user requirements
- To detect defects early and correct them.
- Review typically done using standard inspection process:
 - Checklists.

What is the difference between a System Specification and a Software Specification?

- System specification concerns full functionality of the system.
 - Some of the functionalities may be performed in hardware and some in software.
- Example, if the system is required to move an object to a particular position within an accuracy of ± 1 millimeter:
 - This requirement would be met partly by each of mechanical hardware, servo amplifier and electronics, and the software.

System Specification

- System specification needs to address the following issues:
 - Define the functions of the system
 - Define the Hardware / Software Functional Partitioning
 - Define the Performance Specification
 - Define the Hardware / Software Performance Partitioning
 - Define Safety Requirements
 - Define the User Interface
 - Develop Installation Drawings/Instructions.

Video Rental Store Software

- A video rental store has a large collection of video CDs and DVDs in VHS and MP4 format as well as music CDs.
- A person can become member a member by depositing Rs. 1000 and filling up details such as name, address, and telephone number. A member can cancel his membership and take back his deposit, if he has no dues outstanding against him.
- Whenever the store purchases a new item, details such as price and date of procurement are entered. The daily rental charge is also entered by the manager. After passage of a year, the daily rental charge is automatically halved.
- A member can, at a time, take on loan at most two video CDs and one music CD. The details are entered by a store clerk and a receipt indicating the daily rental charge should be printed by the software.
- Whenever a member returns his loaned item(s), the due amount to be paid is displayed. After the amount is paid, the items are marked returned.
- If a customer loses or damages any item, the full price of the item is charged to him and the item is removed from the inventory.
- If an item is not lent out by anyone for even once over a year, the item is sold at 10% of the purchase price and is removed from the inventory.
- The manager can, at any time, check the profit/loss account.

IIT Security Software

- The staff and students of IIT register their vehicle by filling up a form which a security staff would enter into the system.
- Each time a vehicle enters or leaves the campus, a camera reads the registration number of the car, and the model of the car. If the car is registered, the check gate should lift automatically to let in (or out) the vehicle. The details regarding the entry and exit of the vehicle are registered in the database. For outside vehicles, the driver needs to enter the purpose of entry, the model number, the registration number, and a photograph of the vehicle is stored in the database.
- When an outside vehicle leaves the campus, the details are automatically registered in the database. For any external vehicle that is inside the campus for more than 8 hours, the driver is stopped by the security personnel manning the gate, queried to satisfaction, and the response are recorded. Considering that there have been several incidence of speeding and rough driving, the security personnel are empowered to telephone the registration number of errant vehicles to the main gate. For outside vehicles, the driver is quizzed at the check gate and his future entry is barred if the response is not satisfactory. For inside vehicles, a letter is issued to the dean to deal with the employee or student as the case might be.
- The security officer can check the data regarding the number of vehicles going in and coming out of the campus (over a day, month or year), the total number of vehicles currently inside the campus.

Personal Library System

- The software is required to manage the collection of books by individuals.
- A person can have a few hundreds of books. The details of all the books such as name of the book, year of publication, date of purchase, price, and publisher must be entered. A book is to be given a unique serial number by the computer which would be written by the owner using a pen on the book. Before a friend can be lent a book, he must be registered. The registration data would include name of the friend, address, land line number, and mobile number. Before a friend is issued a book, the various books outstanding against him also with the date borrowed are displayed. The date of issue and the title of the book are stored. When a friend returns a book, the date of return is stored and the book is removed from his borrowing list. Up on query, the software should display the name, address, and telephone numbers of each friend against whom books are outstanding along with the titles of the outstanding books and the date on which those were issued.
- The owner of the library software, when he borrows books from his friends, would enter the details regarding the title of books borrowed, and the date borrowed. Similarly, the return details of books would be entered. The software should be able display all the books borrowed from various friends. The owner should be able to query about the availability of a particular book, the total¹⁰¹ number of books in the personal library,

Travel Agency Automation Software

- The company owns a number of vehicles (cars and SUVs) of various make and intends to deploy a web-based software. A customer can become regular customer by depositing Rs. 5,000/-.
- When a customer requests for a car, he is displayed the types of vehicles are available, and their charges. For every vehicle, there is a per hour charge, and a per kilometer charge. A vehicle can be rented for a minimum of 4 hours. An AC vehicle of a particular category is charged 50% more than a non-AC vehicle of the same category. There is a charge of Rs 300 for every night of halt regardless of the type of the vehicle.
- A customer can book a vehicle, only if he has at least Rs. 2000/- advance. The customer needs to enter when he expects to return the car.
- When the car is returned, depending on the usage, if possible the amount is fully adjusted from the advance, or he has to pay additional amount to cover the cost incurred.
- The company can acquire new vehicles and add them to the fleet of its vehicles. Cars may be condemned and sold off. A car which is currently with the company can be in one of three states: it may have gone for repair, it may be available, it may be rented out. If it is rented out, the company records the date and time when it has been rented out, and the mile-meter reading of the car at that time.
- The company also wants to maintain the maintenance and running expense for each vehicle.
- The manager of the company should be able to query statistics about various types of vehicles, their prices, average expense on repairs for the car, average demand, revenue earned, and fuel consumption of the car.

Students' Auditorium Management Software

- Various types of social and cultural events are conducted in the student auditorium. The auditorium secretary should have the overall authority of scheduling the shows, selecting and authorizing the show managers, as well as sales agents.
- There are two categories of seats: balcony and ordinary seats. Normally balcony seats are more expensive in any show.
- The show manager fixes the prices of these two categories of seats for a specific show, depending on the popularity of a show. The show manager also determines the number of balcony and ordinary seats that can be put on sale. For each show, some seats are offered as complimentary gifts to important functionaries of the students' society and to VIPs which need to be entered into the system.
- The show manager also enters the show dates, the number of shows on any particular date and the show timings. It is expected that the software would support a functionality to let the show manager configure the different show parameters.

- The auditorium secretary appoints a set of sales agents. A sales agent gets a commission of 1% of the total sales he makes for any show.
- The system should let the spectators query the availability of different classes of seats for a show on-line. For the convenience of the spectators, two ways of seat booking are supported. If a spectator pays Rs. 1000/-, a unique 10 digit id is generated and given to him.
- He can use this id to book seats for the shows on-line by using a web browser. For each seat booked for a show using the unique id, he would get a 10% discount. A spectator can also book a seat for a single show only through regular payment.
- For an on-line booking, a spectator would indicate for the type of the seat required by him, the requested seat should be booked if available,
- and the software should support printing out the ticket showing the seat numbers allocated. A spectator should be able to cancel his booking before 3 clear days of the show.
 - In this case, the ticket price is refunded to him after deducting Rs.5/- as the booking charge per ticket.
 - If a ticket is returned at least before 1 day of a show, a booking charge of Rs.10/- is deducted for ordinary tickets and Rs.15/- is deducted for balcony tickets. When a cancellation is made on the day of the show, there is a 50% deduction.

Representation of complex processing logic:

- Decision trees
- Decision tables

Decision Trees

- Decision trees:
 - Edges of a decision tree represent conditions
 - Leaf nodes represent actions to be performed.
- A decision tree gives a graphic view of:
 - Logic involved in decision making
 - Corresponding actions taken.

Example: LMS

- A Library Membership automation Software (LMS) should support the following three options:
 - New member,
 - Renewal,
 - Cancel membership.

Example: LMS

- When the new member option is selected,
 - The software asks details about the member:
 - name,
 - address,
 - phone number, etc.

Example_(cont.)

- If proper information is entered,
 - A membership record for the member is created
 - A bill is printed for the annual membership charge plus the security deposit payable.

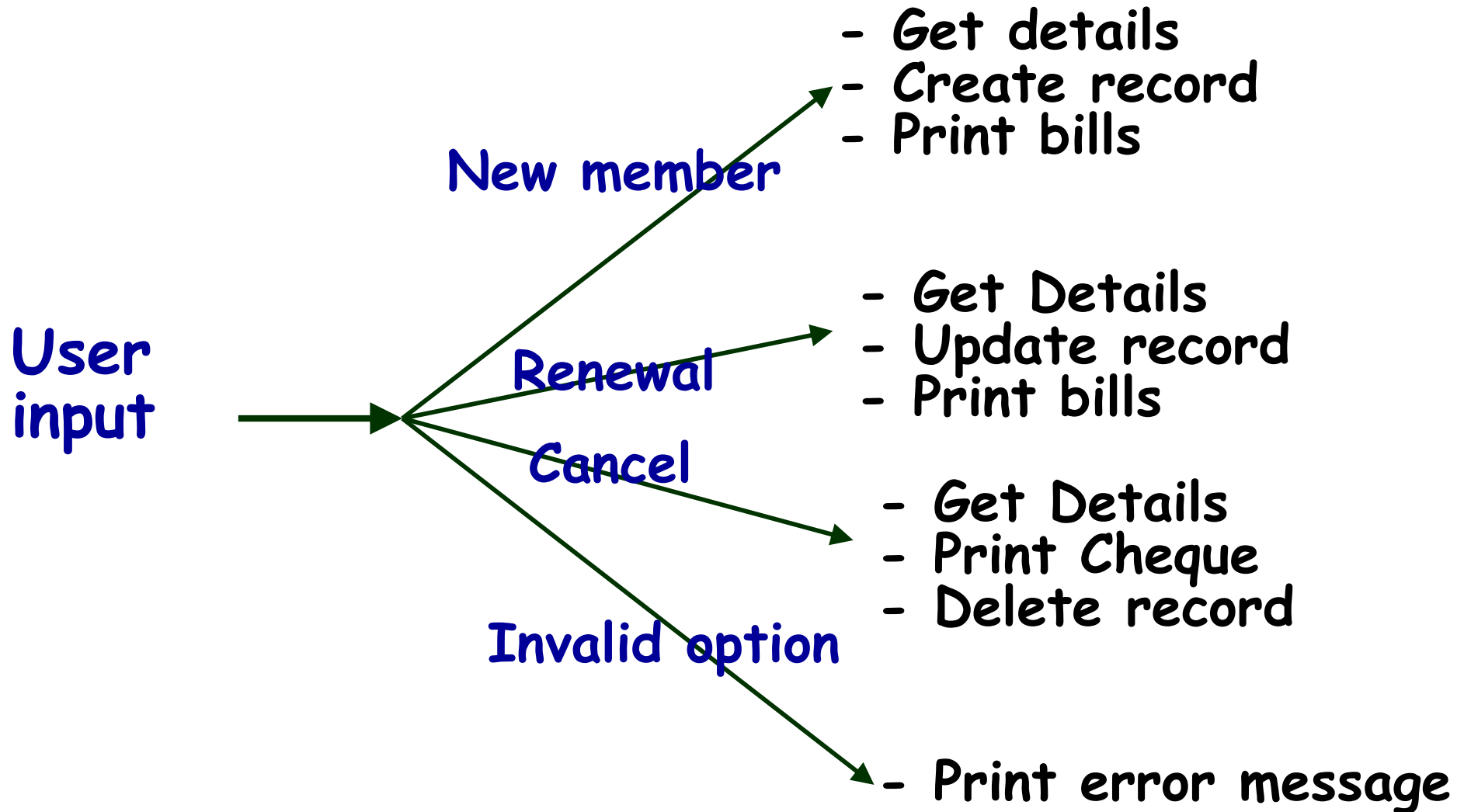
Example_(cont.)

- If the renewal option is chosen,
 - LMS asks the member's name and his membership number
 - checks whether he is a valid member.
 - If the name represents a valid member,
 - the membership expiry date is updated and the annual membership bill is printed,
 - otherwise an error message is displayed.

Example_(cont.)

- If the cancel membership option is selected and the name of a valid member is entered,
 - The membership is cancelled,
 - A cheque for the balance amount due to the member is printed
 - The membership record is deleted.

Decision Tree



Decision Table

- Decision tables specify:
 - Which variables are to be tested
 - What actions are to be taken if the conditions are true,
 - The order in which decision making is performed.

Decision Table

- A decision table shows in a tabular form:
 - Processing logic and corresponding actions
- Upper rows of the table specify:
 - The variables or conditions to be evaluated
- Lower rows specify:
 - The actions to be taken when the corresponding conditions are satisfied.

Decision Table

- In technical terminology,
 - A column of the table is called a rule.
 - A rule implies:
 - If a condition is true, then execute the corresponding action.

Example

- Conditions

Valid selection		NO	YES	YES	YES
New member	--	YES	NO	NO	
Renewal	--	NO	YES	NO	
Cancellation		--	NO	NO	YES

- Actions

Display error message		--	--	--
Ask member's name etc.				
Build customer record	--	--		--
Generate bill	--		--	
Ask membership details		--		
Update expiry date	--	--	--	
Print cheque		--	--	
Delete record	--	--	--	

Comparison

- Both decision tables and decision trees
 - Can represent complex program logic.
- Decision trees are easier to read and understand
 - When the number of conditions are small.
- Decision tables help to look at every possible combination of conditions.

Formal Specification

- A formal specification technique is a **mathematical method** to:
 - Accurately specify a system.
 - Verify that implementation satisfies specification.
 - Prove properties of the specification.

Formal Specification

- Mathematical techniques used include:
 - Logic-based
 - set theoretic
 - algebraic specification
 - finite state machines, etc.

Formal Specification

- Advantages:

- Well-defined semantics, no scope for ambiguity
- Automated tools can check properties of specifications
- Executable specification

Formal Specification

- Disadvantages of formal specification techniques:
 - Difficult to learn and use
 - Not able to handle complex systems

Semiformal Specification

- Structured specification languages
 - SADT (Structured Analysis and Design Technique)
 - PSL/PSA (Problem Statement Language/Problem Statement Analyzer)
 - PSL is a semi-formal specification language
 - PSA can analyze the specifications expressed in PSL

Executable Specification Language

- If specification is expressed in formal language:
 - it becomes possible to execute the specification to provide a system prototype.
- However, executable specifications are usually slow and inefficient.

Executable Specification Language

- Executable specifications only test functional requirements:
 - If non-functional requirements are important for some product,
 - The utility of an executable specification prototype is limited.

4GLs

- 4GLs (Fourth Generation Languages) are examples of
 - executable specification languages.
- 4GLs are successful
 - because there is a lot of commonality across data processing applications.

4GLs

- 4GLs rely on software reuse
 - Where common abstractions have been identified and parameterized.
- Rewriting 4GL programs in higher level languages:
 - Result in upto 50% lower memory requirements
 - Also the programs run upto 10 times faster.

Summary

- Requirements analysis and specification
 - An important phase of software development:
 - Any error in this phase would affect all subsequent phases of development.
- Consists of two different activities:
 - Requirements gathering and analysis
 - Requirements specification

Summary

- The aims of requirements analysis:
 - Gather all user requirements
 - Clearly understand exact user requirements
 - Remove inconsistencies and incompleteness.
- The goal of specification:
 - Systematically organize requirements
 - Document the requirements in an SRS document.

Summary

- Main components of SRS document:
 - Functional requirements
 - Non-functional requirements
 - Constraints
- Techniques to express complex logic:
 - Decision tree
 - Decision table

Summary

- Formal requirements specifications have several advantages.
 - But the major shortcoming is that these are hard to use.

Assignment

- A public library is considering the implementation of a computer-based system to help administer book loans at libraries.
- Identify the stakeholders in such a project.
- What might be the objectives of such a project and how might the success of the project be measured in practical terms?

Stake holders

- Representatives of the local authority who own and pay for the library
- Library staff at various levels
- Technical support staff
- Software and hardware suppliers
- Library users

- **Objectives**
- 'To have in place an operational computer-based loans management system by dd/mm/yy which meets the requirements specified in Annex at a development cost of not more than e.g. Rs.10,00,000.'
- **Measures of effectiveness**
- Was the application actually working by dd/mm/yy?
- Were costs under £xxxx?
- Can users etc certify that delivered system meets the documented requirements?